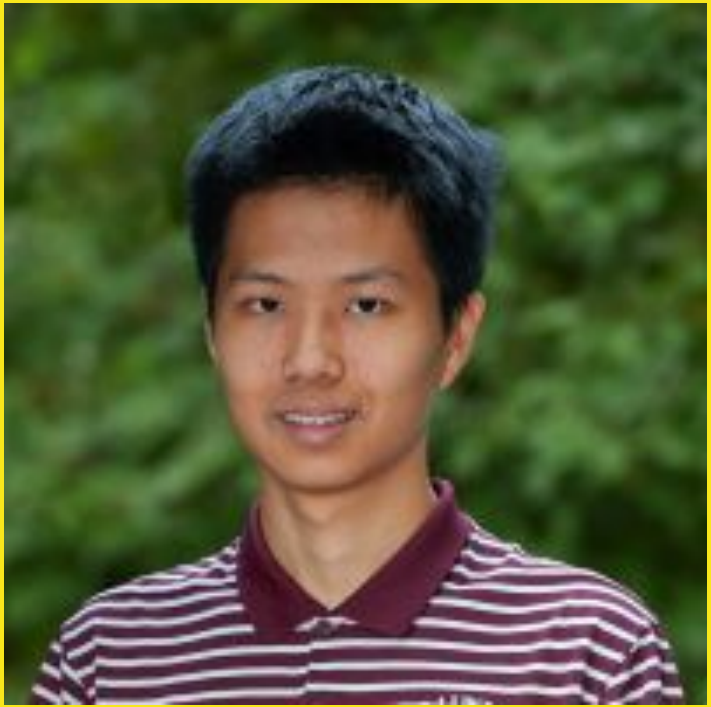
Welcome to the Artificial Intelligence Workshop (Week 2)

Let's Learn How Computers Learn from Information

About Me



Name: Peikang Hu

Current Education: Rising senior at St. Luke's School, New Canaan, Connecticut

Experiences in coding(Python, C++, Java..) and machine learning

Icebreaker

How Many AI Tools Have You Used?

Raise your hand if you've used...

- □ ChatGPT
- □ Siri or Alexa
- □ Google Translate
- □ YouTube recommendations
- □ TikTok or Instagram
- □ Face ID on a phone

Did you know?

You've probably been using AI almost every day!



Workshop Overview

Week 1 – What is Artificial Intelligence (AI)?

Goal: Spark curiosity and understand what AI is.

Week 2 – How Computers Learn

Goal: Introduce linear regression and its application.

Week 3 – A Continuation of Week 2 BUT with more Fun!

Goal: Introduce more advanced AI models.

Week 4 – A Continuation of Week 3 BUT with more Fun!

Goal: Introduce more advanced AI models.

Learning Objectives

By the End of Today, You Will Be Able To...

✅ Understand what a **Linear Regression** model is and what problems it can solve.
✅ Build your first **Linear Regression** model using Python.
✅ Train a model to make predictions on unseen data.
✅ Evaluate your model's performance using basic metrics.
✅ Interpret the relationship between input features and predictions.

Review: Machine Learning

What Is Machine Learning?

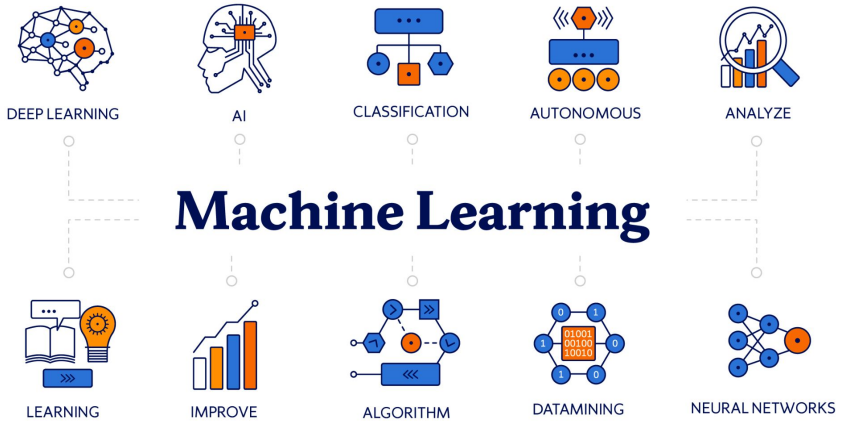
Machine Learning = Teaching computers to learn from examples instead of giving every rule.

Traditional Programming:

Human writes rules
Computer follows rules

Machine Learning:

Give examples
Computer finds patterns



Syracuse University
School of Information Studies

Three Types of Machine Learning

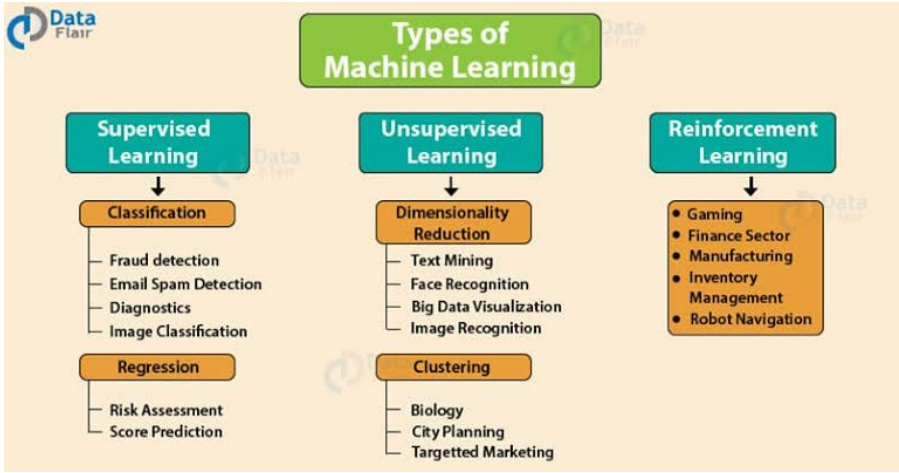
Supervised Learning

Data has **inputs AND correct answers.**

Example: Predicting Exam Scores

Imagine we collect data from several students:

- A student who studied **2 hours** scored **65**.
- A student who studied **4 hours** scored **80**.
- A student who studied **6 hours** scored **92**.
- A student who studied **8 hours** scored **97**.



Using many examples like these, a machine learning model learns the relationship between **hours studied** and **exam score** so it can predict the score for a new student.

Three Types of Machine Learning

Unsupervised Learning

Only inputs.

No labels.

Example:
Group customers based on shopping habits.

Output:
Different customer clusters.



Three Types of Machine Learning

Reinforcement Learning

An agent learns through rewards and penalties.

Examples:

- Chess
- Robotics
- Video games



Today's Highlight:

Supervised Learning!!!

Two Types of Supervised Learning

Classification

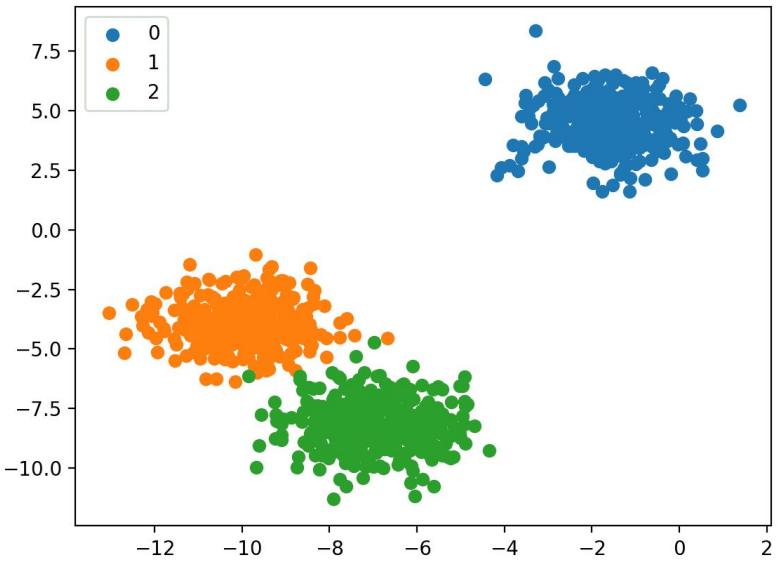
Predict categories.

Examples

- Spam or not spam
- Dog or cat
- Fraud or legitimate

Output:

Discrete labels



Two Types of Supervised Learning

Regression

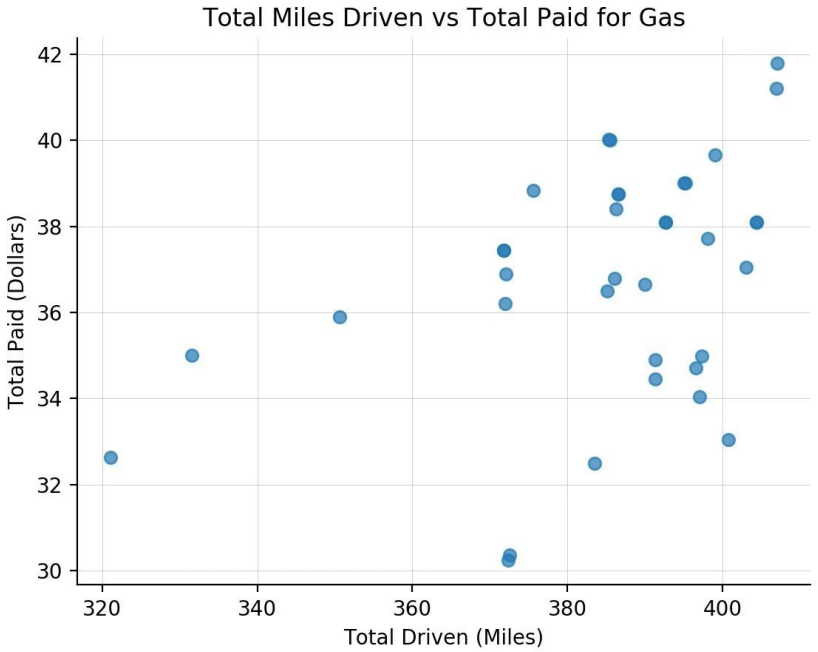
Predict numbers.

Examples

- House prices
- Temperature
- Stock prices
- Salary

Output:

Continuous values



Today's topic:

Linear Regression!!!

What is Linear Regression?

Linear Regression is the **simplest regression algorithm.**

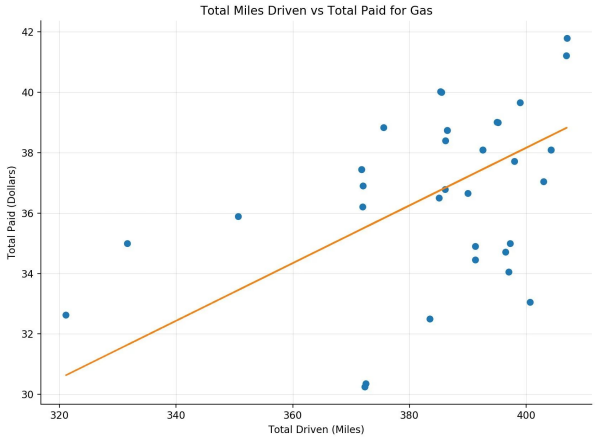
Goal:

Find the best straight line through data.

Example:

Hours Studied → Exam Score

More studying generally leads to higher scores.



The computer finds the line that best describes this relationship.

Why Does Linear Regression Work?

Many real-world relationships are approximately linear.

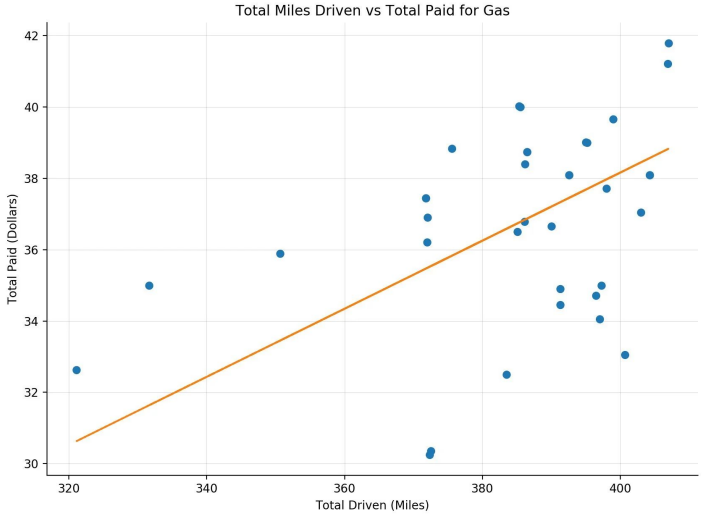
Examples

- Height vs Weight
- Years of Experience vs Salary
- Advertising Budget vs Sales
- Engine Size vs Fuel Consumption

Linear Regression assumes

> As one variable changes,
> another changes approximately in a straight-line manner.

Not every relationship is linear—but many are close enough.



The Equation of a Line(Algebra)

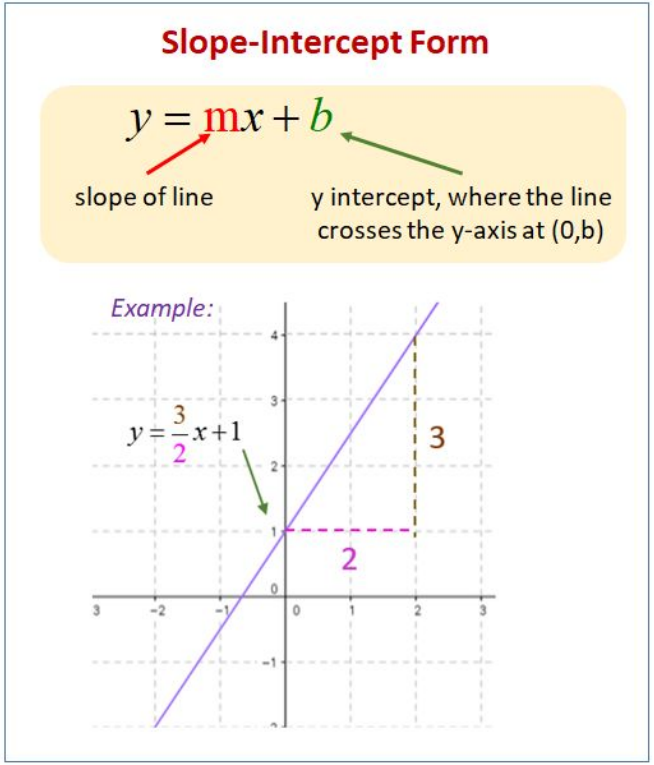
Review from Algebra

The equation of a line is:

y = mx + b

Where:

- **x** = the input (also called the **independent variable**)
- **y** = the output (the value we want to predict)
- **m** = the **slope** of the line (how much y changes when x i by 1)
- **b** = the **y-intercept** (the value of y when x = 0)



The Equation of a Line(Machine Learning)

Linear Regression uses almost the same equation:

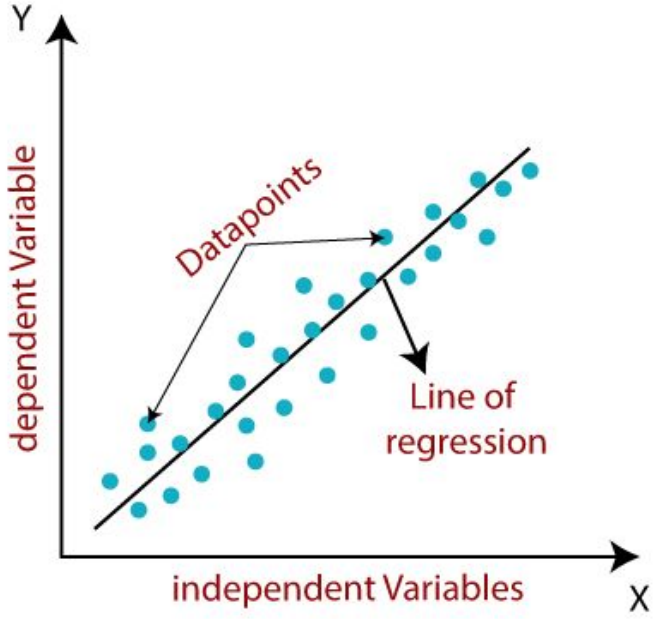
Prediction = Weight × Feature + Bias

or

$\hat{y} = wx + b$

Where:

- **x** = input feature (e.g., hours studied)
- **$\hat{y}$** = predicted value (e.g., predicted exam score)
- **w** = weight (same idea as the slope)
- **b** = bias (same idea as the y-intercept)



Key Idea: Linear Regression is essentially finding the **best possible line** that describes the relationship between the input and the output.

Understanding Parameters

Suppose

$y = 5x + 40$

Interpretation:

Every extra hour studied increases the predicted exam score by **5 points**.

If someone studies 0 hours

Predicted score = 40

Slope tells us

How quickly predictions increase.

Intercept tells us

Where the line begins.

How Does the Computer Find the Best Line?

Imagine drawing many different lines.

Some fit well.

Some fit poorly.

Question:

How does the computer know which one is best?

Answer:

Measure the errors.

The best line is the one with the **smallest overall error.**

Residuals (Prediction Errors)

Residual = Actual Value − Predicted Value

Example:

- Actual Score = **90**
- Predicted Score = **85**
- **Residual = 90 − 85 = 5**

Key Idea

- A residual measures how far a prediction is from the actual value.
- **Smaller residuals = Better model**

Mean Squared Error (MSE)

Instead of adding errors,

we square them.

Why square?

Positive and negative errors don't cancel.

Large mistakes receive larger penalties.

Goal:

Find the line with the lowest MSE.

$$\text{MSE} = \frac{1}{n}\sum(Y_i - \hat{Y}_i)^2$$

Implementation of Linear Regression

1. Import the necessary libraries

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
```

Explanation: Importing NumPy for numerical operations, Matplotlib for visualization, and Scikit-learn for building the linear regression model.

Implementation of Linear Regression

2. Generating Random Dataset

```
np.random.seed(42)

X = np.random.rand(50, 1) * 100

Y = 3.5 * X + np.random.randn(50, 1) * 20
```

- Sets the **random seed** to 42.
- This makes the random numbers the **same every time** you run the code, so your results are reproducible.

Implementation of Linear Regression

2. Generating Random Dataset

```
np.random.seed(42)

X = np.random.rand(50, 1) * 100

Y = 3.5 * X + np.random.randn(50, 1) * 20
```

- `np.random.rand(50, 1)` generates **50 random numbers** between **0 and 1**.
- `50` = number of data points.
- `1` = one feature (one column).
- Multiplying by `100` scales the values to the range **0 to 100**.
- These are your **input (independent variable)** values.

Implementation of Linear Regression

2. Generating Random Dataset

```
np.random.seed(42)

X = np.random.rand(50, 1) * 100

Y = 3.5 * X + np.random.randn(50, 1) * 20
```

- `3.5 * X` represents the true relationship, meaning that for every 1-unit increase in `X`, `Y` increases by about **3.5**.
- `np.random.randn(50, 1)` generates 50 random values from a normal (Gaussian) distribution with mean **0** and standard deviation **1**.
- **Multiplying by** `20` increases the amount of noise, making the data more realistic instead of perfectly following a straight line.
- The final equation is approximately: `Y = 3.5X + random noise`, where the noise represents natural variation or measurement error.

Implementation of Linear Regression

3. Creating and Training Linear Regression Model

```
model = LinearRegression()
model.fit(X, Y)
```

Explanation: Initialize the model and train it using the generated dataset.

Implementation of Linear Regression

4. Predicting Y Values

```
Y_pred = model.predict(X)
```

Explanation: Using the trained model to predict output values for the input data.

Implementation of Linear Regression

5. Visualizing the Regression Line

```
plt.figure(figsize=(8,6))
plt.scatter(X, Y, color='blue', label='Data Points')
plt.plot(X, Y_pred, color='red', linewidth=2, label='Regression Line')
plt.title('Linear Regression on Random Dataset')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.grid(True)
plt.show()
```

Explanation: Plot the original data points along with the best-fit regression line.

Slope and Intercept

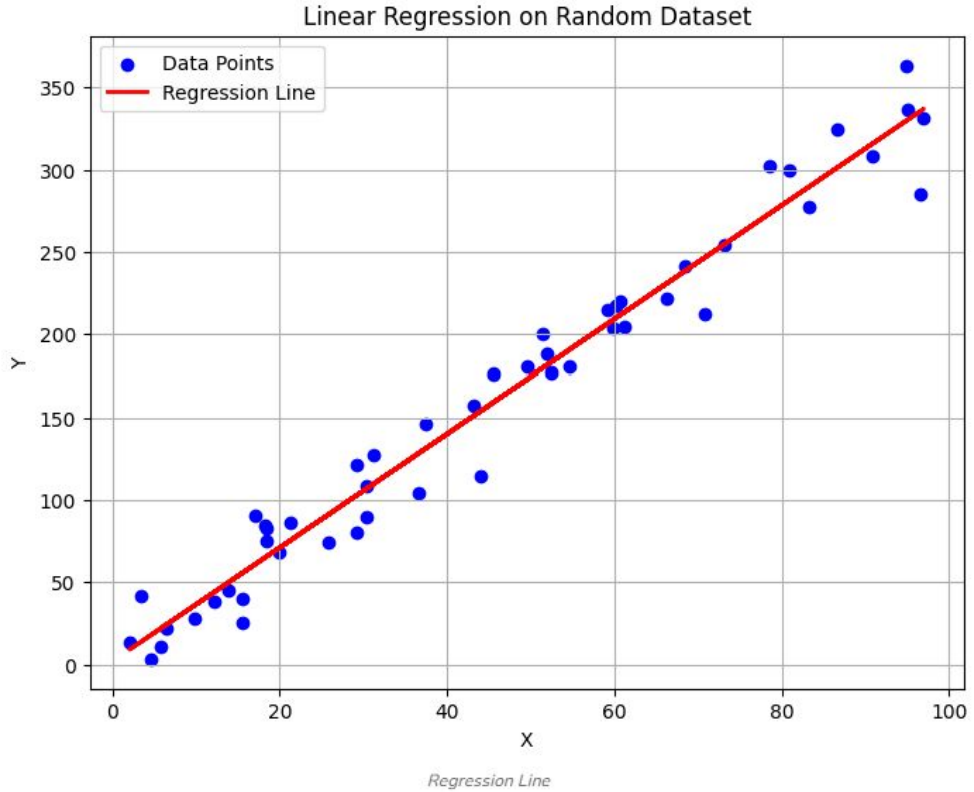
```
print("Slope (Coefficient):", model.coef_[0][0])
print("Intercept:", model.intercept_[0])
```

Explanation: Print the learned slope and intercept values of the regression model.

Output

- **Blue dots** represent the actual data points in the dataset.
- **Red line** is the **linear regression line** (best-fit line) that models the relationship between **X** and **Y**.

Output:

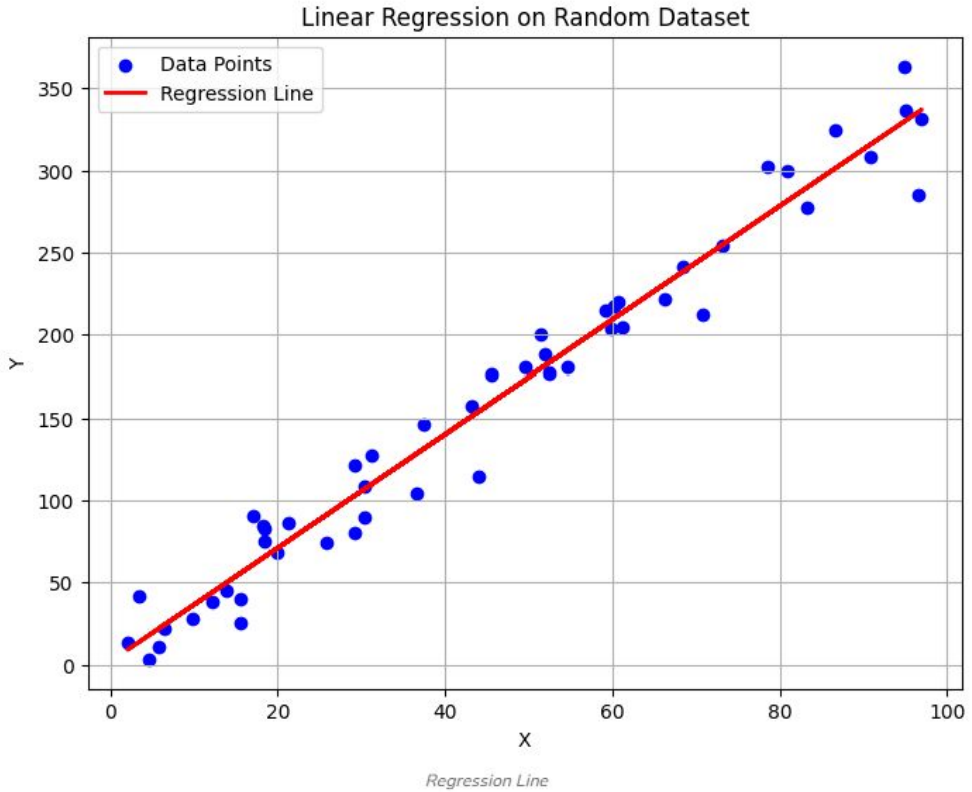


Regression Line

Output

- The graph shows a **strong positive linear relationship**: as **X increases, Y also increases**.
- Most data points lie **close to the regression line**, indicating the model fits the data well.
- Some points are **above or below the line** due to **random noise (variation)** added to the dataset.
- The **vertical distance** between a data point and the regression line is called the **residual (prediction error)**.
- **Smaller residuals** mean the model's predictions are more accurate.

Output:



Regression Line

Output

```
Slope (Coefficient): 3.4553132007706204
Intercept: 1.9337854893777546
```

Slope (Coefficient): 3.4553

- The slope indicates how much **Y is expected to increase** for every **1-unit increase in X**.
- Here, for every increase of **1** in **X**, **Y increases by about 3.46** on average.
- A **positive slope** means there is a **positive relationship** between X and Y.
- Since the original data was generated with a true slope of **3.5**, the model's estimate (**3.4553**) is very close.

Output

```
Slope (Coefficient): 3.4553132007706204
Intercept: 1.9337854893777546
```

Intercept: 1.9338

- The intercept is the **predicted value of Y when X = 0**.
- In this model, if **X = 0**, the predicted **Y is about 1.93**.
- The intercept serves as the **starting point** of the regression line on the Y-axis.
- It is close to **0**, which makes sense because the generated data did not include a large constant offset.

Pros and Cons of Linear Regression Models

Pros

- Simple and easy to use
- Fast to train
- Easy to interpret
- Works well for linear data
- Predicts continuous values

Cons

- Only models linear relationships
- Sensitive to outliers
- Can underfit complex data
- Assumes linearity
- Struggles with correlated features

Preview of Next Week!

Next Week: Can Computers Learn More Than a Straight Line?

- What if the relationship isn't linear?
- Meet **Decision Trees** – models that make predictions by asking a series of questions.
- Discover how AI learns complex patterns that linear regression can't.




COME TO THE AI
WORKSHOP NEXT WEEK
on August 8!!!!!!!!